

LegalShield 総合状況報告書

作成日：2026 年 5 月 31 日 作成：劉 建志 + Cascade 目的：現在の到達点・今セッションの成果（幻覚防止ハーネス）・将来の運用方法・全重要情報の一括把握

0. エグゼクティブサマリー

LegalShield は、市民が直面する 12~27 種類の法的・社会的問題について、**端末の問診から最適な相談先・法的根拠へ導く公益 GIS / triage プラットフォーム**です。

本日の最重要成果は、「**データベースは完璧なのに AI が幻覚で答える**」根本問題を構造的に解決したことです（Anti-Hallucination Harness L1-L7 実装）。

領域	状態
triage MVP（6問問診 → 相談先）	✅ 稼働（71 routes / 12 categories / DV 328件）
FEWN 暗号マッチング	✅ CLI デモ完成・動作確認済
音声トリアージ・匿名化・Q-Map	✅ 実装（iOS は Mac でビルド要）
幻覚防止ハーネス（本日）	✅ バックエンド完成・テスト合格
仮名化 2 層 DB（Vault/Operating）	🔧 設計完了・実装一部
管理者ダッシュボード	❌ 未着手
RISTEX 提案書 v7.5	✅ 完成（6/3 締切・教授承諾待ち）

1. 今セッションの最重要成果：幻覚防止ハーネス

1-1. 何が問題だったか

「DB は完璧なのに、AI がそれを見ずに“それっぽい嘘”を自信満々に答えていた」

診断の結果、原因は DB でも検索精度でもなく、**アプリの配線**でした。

- iOS アプリのチャットが、検索なしの生 LLM（`/api/generate`）を直接叩いていた
- AI は記憶だけで回答し、条文番号・判例・金額を**創作**できる状態だった
- バックエンドには正しい検索付き回答 `/rag/query` があったが、**誰も呼んでいなかった**

1-2. どう直したか（ロジック）

「賢い AI」ではなく、**AI が嘘をつけない“構造”**を作りました（Harvard 系リーガルテックと同思想）。

質問

- ① 質問の種類・リスクを判定（時効/署名など不可逆 → 自動で弁護士必須）
- ② 必ず先にDB検索（しないと先へ進めない＝検索ゲート）
- ③ AIに「引いた資料だけで答えろ、各文に出典[S1]を付けろ」と強制
資料が無ければ「答えられない」と言う（嘘より正直）
- ④ 自己チェック：回答中の条文番号・金額が資料に実在するか機械照合
無ければ「未裏付け」と赤旗を立て信頼度を下げる
- ⑤ 画面に必ず表示：出典・信頼度(5段階)・弁護士サイン・未裏付け警告
- ⑥ 全質問/回答を改ざん不可能な記録(ハッシュ連鎖)で保存

1-3. テストで確認済みの挙動

入力	システムの反応
正しい根拠付き回答	信頼度 5/5 、未裏付けゼロ
「民法第999条で1000万円が確定」（資料に無い創作）	自動検出 → 未裏付け赤旗・信頼度0.4・弁護士フラグ点灯
検索ヒットゼロ	AI が 回答拒否 → 専門窓口へ案内（信頼度0.0）
「時効前に和解契約に署名すべき？」	不可逆判断 と認識 → 弁護士相談を強制表示

1-4. 【重要】将来どう運用するか

A. 通常運用（iOS / 外部 agent） - 法律質問はすべて **POST /rag/answer** を呼ぶ。生の **/api/generate** は法律質問に使わない。 - iOS は **LegalHarnessService** → **/rag/answer** → **HarnessAnswerView** で出典付き回答を表示。 - バックエンド起動：`.venv` で `uvicorn legalshield.backend.api:app --host 0.0.0.0 --port 8000`

B. 精度を上げたいとき - リクエストに **judge_model** を指定すると、**別の LLM が二次検証**（クロスチェック / L5）。 - **top_k** で根拠数を調整。 **use_statutes=false** で判例のみグライディング（statutes テーブル未整備時）。

C. 監査・説明責任 - 全 Q&A が **lancedb/harness_audit.jsonl** に **SHA-256 連鎖**で記録される。改ざん検知・第三者説明に使える。

D. 拡張（将来フェーズ） - 設計書 L3「変数群」（行政トポロジ・予算サイクル・行動経済学モデル）を案件タイプ別に pre-load → 推論の文脈を厚くする。 - エッジ 3B SLM を L1 トリアーに接続（下記 §4）。

1-5. 実装ファイル

バックエンド（Python・テスト合格） - **legalshield/backend/harness.py** — L1～L7 本体（依存注入型・オフラインテスト可能） - **legalshield/backend/api.py** — **POST /rag/answer** 追加（既存エンドポイントは無改変）

iOS（Swift・Mac で `xcodegen generate` → 実機ビルド要） -

Services/HarnessModels.swift — レスポンス型 -

Services/LegalHarnessService.swift — **/rag/answer** クライアント -

Services/LLMService.swift — **legalQA** を検索ゲート経由に配線（失敗時のみ生 LLM へフォールバックし明示） - **Views/HarnessAnswerView.swift** — 透明性 UI

2.9 節点パイプラインの現状

[1]録音 → [2]ASR → [3]暗号封印 → [4]On-device分類
→ [5]雲端LLM → [6]暗号マッチング(FEWN) → [7]遠隔送信
→ [8]管理者ダッシュ → [9]公共訴訟ブリッジ

#	節点	状態	残るGAP
1	録音	✓ 4モード	Wake-word・SNR
2	ASR	✓ WhisperKit+学習基盤	訓練データ・話者分離
3	暗号封印	✓ SHA-256 chain	● TSA タイムスタンプ局連携
4	On-device分類	✓ MLX+taxonomy	モデルDL・評価セット
5	雲端LLM	✓ Bedrock+本日 harness	injection防御・レート制限
6	FEWNマッチング	✓ CLIデモ完成	PSI本番化・iOSクライアント
7	遠隔送信	✓ schema	再送キュー・E2E暗号
8	管理者ダッシュ	✗ 未着手	Web UI・RBAC
9	公共訴訟ブリッジ	✗ 未着手	● CALL4形式export

3.5 ロール利用アーキテクチャ + 情報フロー

ロール	見える情報	見えない情報
使用者（被害当事者）	自分の相談・回答・出典・自分の証拠	他人の一切
導入者（自治体・NPO）	地域の 集計のみ （ $k \geq 5$ ）・リスクマップ	個人特定情報・生データ
協力者（CALL4・弁護団）	FEWN暗号マッチ結果・ 本人同意済み 案件サマリ	顔・電話・住所（2-of-N同意要）
管理者（運用責任者）	監査ログ・稼働状況・ 仮名 統計	生PII（復元には2-of-N決議）
開発者（劉）	全コード・スキーマ・テストデータ	本番Vault実データ （Review→AWS反映）

情報ストリーム

使用者（端末）
└ エッジSLMでトリアージ → 生データは端末内AES-GCM暗号化
└ 送信は分離：トークン + 去識別化trriage JSON のみ
└
└ ▼ 主DB = 物理2分離
└ ① IDENTIFIER VAULT（HSM/KMS, 別VPC） ← 管理者も直接見れない / 復元=2-of-N+
└ ② OPERATING DB（公開・連携・分析） ← /rag/answer・FEWN・Q-Map集計はここが

各ロールへは「同意ラダー T0~T4」で段階的に配信

開発 → 本番の流れ（劉のReviewが必須ゲート）

ローカル開発 → `git push(feature)` → 協力者がlocal repoで同期+PR
→ ★劉がReview★ → `main merge` → GitHub Actions → AWS

4. エッジ小型モデル訓練計画

本機（Windows + RTX 4080 16GB）実測

- `.venv` : torch 2.6.0+cu124 (CUDA有効) ・ transformers 5.8.0 ✓
- 不足 : `peft` `trl` `datasets` `accelerate` (微調整前に install)
- WSL2 Ubuntu 利用可 (QLoRA/bitsandbytes 用) ・ Ollama 起動中

2層モデル構成

エッジ 3B (Qwen2.5-3B 第一候補) = トリアージ・構造化
| 去識別化JSON

主DB側 gemma3:27b = /rag/answer で grounded 本格分析

段階

- **Phase A (今すぐ)** : `ollama pull qwen2.5:3b` でプロンプト運用 (訓練不要)
- **Phase B (データ蓄積後)** : taxonomy + seed_queries + intake ログ → JSONL → fp16 LoRA 微調整
- **Phase C (精度確認後)** : LoRA マージ → GGUF → `ollama create` → 配信 (Mac は MLX/CoreML 変換)

訓練は「必須」ではなく「改善」。harness の L1 はルールベースで既に動作中。

5. 障害分離設計（小バグで全体が死なない）

Graceful Degradation（縮退運転）を全層で徹底。harness は既にこの思想で実装済み。

フォールバックチェーン:

- | | |
|-------------------------------|--------------------------|
| ① /rag/answer (grounded・出典付き) | ← 通常 |
| ② 生LLM + 「未検証」警告 | ← クラウド生存時 |
| ③ エッジ3B (オフライン分類) | ← ネット断 |
| ④ 静的FAQ + 緊急番号(110/189/#8008) | ← 最終防衛線 (DB/LLM非依存・常時表示) |

- 検索失敗 → 拒否 (幻覚しない) / judge失敗 → 記録のみ / 監査失敗 → 回答は返す (各層 try/except 局所catch済)
- 2バックエンド完全独立 (私有GPU ⊥ 公開Docker) : 片方が落ちてても他方生存

- Vault ⊥ Operating DB（別VPC/別RDS）：片方侵害でも他方無事

6. 締切と次の一手

日付	イベント	残り
6/2(火)	CALL4 谷口共同代表 面談	2日
6/3(水)	JST RISTEX 締切	3日
6/5(金)	トヨタ財団 締切	5日

本機で今すぐできる候補

優先	タスク	コマンド/成果物
すぐ	エッジ3Bを運用	<code>ollama pull qwen2.5:3b</code>
小	訓練ライブラリ導入	<code>.venv</code> に <code>peft trl datasets accelerate</code>
中	<code>/rag/answer</code> 本番ライブ検証	<code>.venv</code> で <code>uvicorn</code> 起動 + 実クエリ
中	CALL4形式export（節点9）	case → 公開訴訟記事 markdown
大	管理者ダッシュボード（節点8）	gis 側 Web UI + RBAC

7. 重要ドキュメント一覧

ファイル	内容
<code>docs/AGENT_SKILL_BOUND_DESIGN.md</code>	幻覚防止 L1-L7 設計書（理論）
<code>docs/strategy/2026-05-31_harness_report.md</code>	ハーネス 実装報告 （CALL4 説明用）
<code>docs/strategy/2026-05-31_edge_training_and_role_architecture.md</code>	訓練・5 ロール・ 障害分離 設計
<code>docs/strategy/DATA_FLOW_PSEUDONYMIZATION.md</code>	仮名化2 層DB設計
<code>docs/strategy/NODE_HARDENING_20260526.md</code>	9節点 GAP + CALL4戦

ファイル	内容
	略 + FEWN仕様
docs/grants/ristex_solve_2026/proposal_draft/SOLVE2026_master_draft_v7_5.md	RISTEX 提案書 最終版 (92点)

8. 一言まとめ

本日、「DBを見ずに幻覚する」根本問題を構造的に解消しました。今後すべての法律QAは検索ゲートを通り、出典・信頼度・弁護士サインが必ず可視化されます。残るは現場データでのライブ検証、CALL4連携（節点9）、管理者ダッシュボード（節点8）です。

— Generated by LegalShield Document Pipeline / 2026-05-31_STATUS_REPORT.md —